IT4010 Research Project – 2026 Jul

Topic Assessment Form

V2.2

Project ID: J26-SE-335

1. Topic (12 words max)

Self-Healing AI-Driven Failure Prediction and Automated Recovery Framework for Microservices

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	
Year	

5. Brief description of the research problem including references (300 words max) – references not included in word count.

Modern enterprise applications are increasingly built as microservices to achieve scalability and agility. However, this distributed architecture introduces significant operational complexity. The interconnected nature of microservices means a failure in one service can rapidly cascade, leading to widespread system degradation or complete outages. The primary users affected by this problem are Site Reliability Engineers (SREs), DevOps teams, and software developers who are burdened with the manual, reactive task of identifying and resolving failures.

The impact of this problem is substantial:

- ***Technical:** Increased system downtime (MTTR), degraded user experience, and wasted computational resources.*
- ***Economic:** Direct revenue loss from service unavailability and high operational costs for manual incident response.*
- ***Social:** Erosion of user trust and satisfaction when applications are unreliable.*

*This research aligns with the **SST (Software Systems & Technologies)** cluster and falls under the **Distributed Systems** and **Cloud Computing** verticals.*

*Furthermore, the project directly contributes to **Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure)** by building resilient and reliable technological infrastructure, which is a foundation for economic growth and innovation.*

*The core problem is that current monitoring tools are reactive (prometheus, grafana), while orchestration platforms like Kubernetes only offer reactive self-healing (restarting failed containers). There is a critical gap in a framework that can **proactively** predict failures and automatically execute intelligent recovery actions at the application layer.*

References:

[1] D. Ganesan et al., "Microservices: A review of concepts, challenges, and solutions," IEEE Access, vol. 8, pp. 1-1, 2020.

[2] J. Soldani, D. A. Tamburri, and W.-J. Van Den Heuvel, "The pains and gains of microservices: A systematic grey literature review," Journal of Systems and Software, vol. 146, pp. 215-232, 2018.

[3] Kubernetes Documentation, "Self-healing applications," 2024. [Online]. Available: https://kubernetes.io/

6. Brief description of Existing Research and Systems (300 words max)

Existing solutions fall into three main categories, each with limitations:

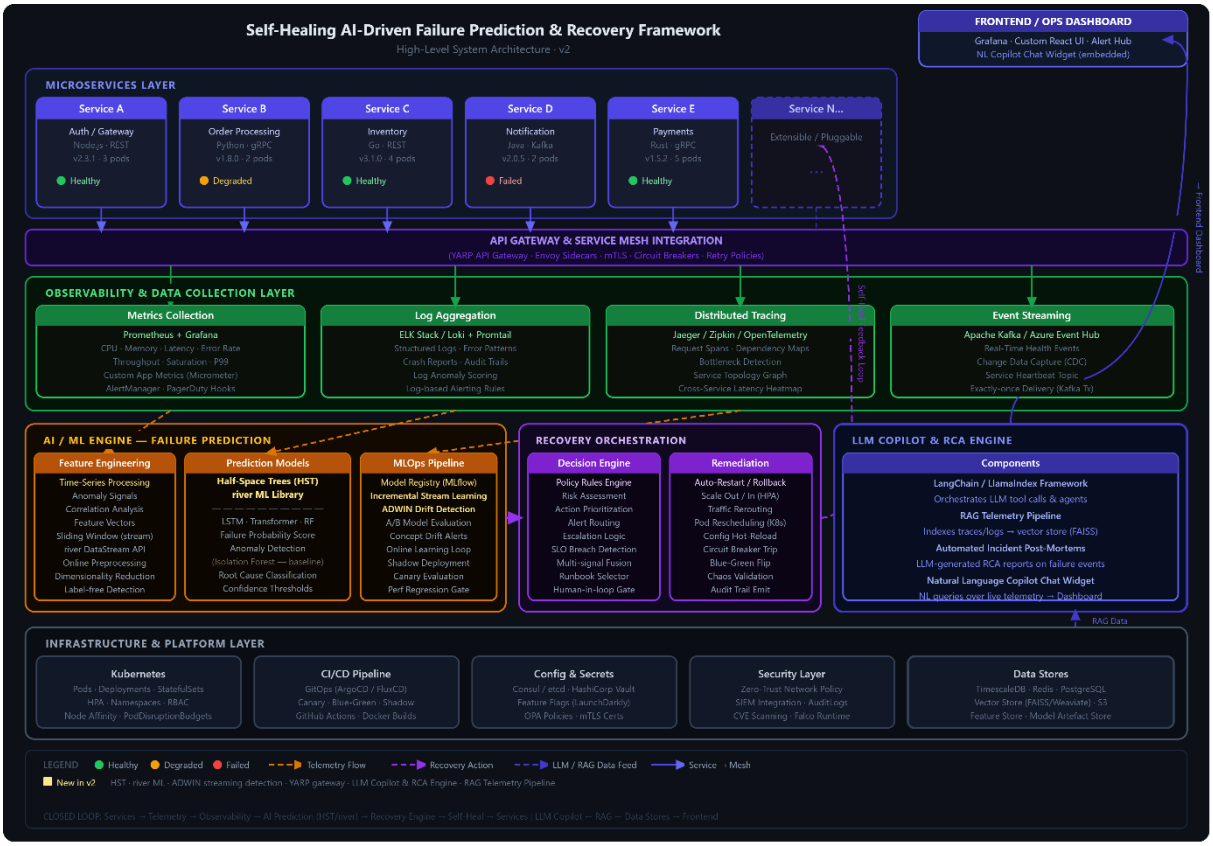
1. ***Monitoring and Observability Tools:*** *Tools like* ***Prometheus****,* ***Grafana****, and* ***Datadog*** *are excellent at collecting, visualizing, and alerting on system metrics. However, they are fundamentally reactive. They inform engineers that a problem is occurring or has occurred but lack the capability to predict future failures or take automated corrective action.*
2. ***Orchestration Self-Healing: Kubernetes*** *provides built-in self-healing mechanisms like restarting crashed containers, rescheduling pods on healthy nodes, and performing rolling updates. While essential, this is a reactive, infrastructure-level response. It often cannot handle more subtle application-level failures like a "degraded" service that is running but returning high error rates or experiencing memory leaks. By the time Kubernetes acts, the failure has already happened and may have impacted users.*
3. ***AI for IT Operations (AIOps) Platforms:*** *Commercial platforms like* ***Dynatrace*** *and* ***Datadog*** *are beginning to incorporate AI for anomaly detection and root cause analysis. However, these are often proprietary, "black-box" solutions that are expensive and not easily customizable. More importantly, their integration with orchestration layers like Kubernetes for automated recovery is often limited or highly vendor-specific.*

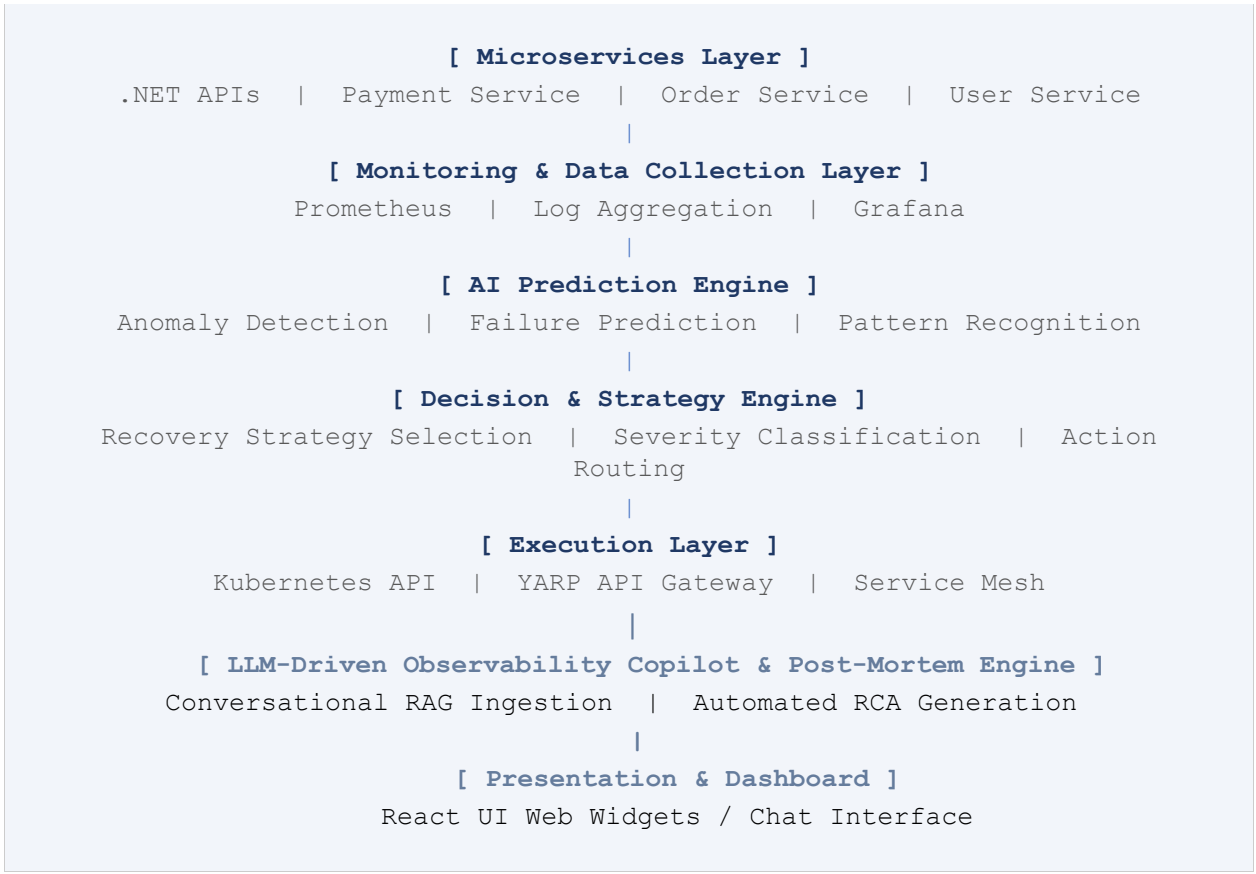
A significant gap persists in the form of an open, extensible framework that integrates ***predictive AI*** *(not just reactive or descriptive) with an* ***intelligent decision engine*** *that can trigger a variety of application and infrastructure-level recovery strategies based on the specific type and severity of the predicted failure. This proposal aims to fill that gap.*

7. Brief description of the solution's nature, including a conceptual diagram (maximum 500 words).

Our proposed solution is a self-healing middleware framework that sits between the observability layer and the Kubernetes orchestration layer. It aims to transform microservice management from reactive failure handling to proactive prevention. The framework directly addresses the gaps identified above by integrating predictive AI with automated, context-aware recovery actions.

- ***Input:*** *The system continuously ingests real-time telemetry data. This includes:*
- ***Metrics:*** *CPU, memory, network I/O, request rates, error rates, latency percentiles from Prometheus.*
- ***Logs:*** *Structured application logs and exception traces.*
- ***Dependency Data:*** *The health and state of inter-service communication.*
- ***Process:*** *The input data is processed in a pipeline:*
- ***AI Prediction Engine:*** *This core component uses machine learning models trained on historical data to:*
- *Detect anomalies in real-time using models like Isolation Forests.*
- *Predict the likelihood of failure in a given service within a specific time window using models like LSTMs or Random Forests.*
- ***Decision & Strategy Engine:*** *This receives the prediction (service ID, failure probability, predicted type). It acts as an intelligent policy layer, consulting a rule-based decision matrix to select the most appropriate recovery strategy. For instance, a predicted memory leak will trigger a pod restart, while a predicted traffic spike will trigger a horizontal scale-out.*
- ***Output:*** *The output is the execution of a targeted recovery strategy.*
- ***Execution Layer:*** *This layer translates the decision from the strategy engine into concrete API calls to external systems to enact the recovery, such as:*
- *Updating the number of replicas for a Kubernetes Deployment.*
- *Directly restarting a specific unhealthy pod.*
- *Modifying routing rules in the YARP API Gateway to divert traffic away from a struggling service.*


Self-Healing AI-Driven Failure Prediction & Recovery Framework
High-Level System Architecture · v2
FRONTEND / OPS DASHBOARD
Grafana · Custom React UI · Alert Hub
NL Copilot Chat Widget (embedded)
MICROSERVICES LAYER
Service A
Auth / Gateway
Node.js · REST
v2.3.1 · 3 pods
Healthy
Service B
Order Processing
Python · gRPC
v1.8.0 · 2 pods
Degraded
Service C
Inventory
Go · REST
v3.1.0 · 4 pods
Healthy
Service D
Notification
Java · Kafka
v2.0.5 · 2 pods
Failed
Service E
Payments
Rust · gRPC
v1.5.2 · 5 pods
Healthy
Service N...
Extensible / Pluggable
API GATEWAY & SERVICE MESH INTEGRATION
(YARP API Gateway · Envoy Sidecars · mTLS · Circuit Breakers · Retry Policies)
OBSERVABILITY & DATA COLLECTION LAYER
Metrics Collection
Prometheus + Grafana
CPU · Memory · Latency · Error Rate
Throughput · Saturation · P99
Custom App Metrics (Micrometer)
AlertManager · PagerDuty Hooks
Log Aggregation
ELK Stack / Loki + Promtail
Structured Logs · Error Patterns
Crash Reports · Audit Trails
Log Anomaly Scoring
Log-based Alerting Rules
Distributed Tracing
Jaeger / Zipkin / OpenTelemetry
Request Spans · Dependency Maps
Bottleneck Detection
Service Topology Graph
Cross-Service Latency Heatmap
Event Streaming
Apache Kafka / Azure Event Hub
Real-Time Health Events
Change Data Capture (CDC)
Service Heartbeat Topic
Exactly-once Delivery (Kafka Tx)
Self-heal restart cmds
Frontend dashboard
AI / ML ENGINE — FAILURE PREDICTION
Feature Engineering
Time-Series Processing
Anomaly Signals
Correlation Analysis
Feature Vectors
Sliding Window (stream)
river DataStream API
Online Preprocessing
Dimensionality Reduction
Label-free Detection
Prediction Models
Half-Space Trees (HST)
river ML Library
LSTM · Transformer · RF
Failure Probability Score
Anomaly Detection
(Isolation Forest — baseline)
Root Cause Classification
Confidence Thresholds
MLOps Pipeline
Model Registry (MLflow)
Incremental Stream Learning
ADWIN Drift Detection
A/B Model Evaluation
Concept Drift Alerts
Online Learning Loop
Shadow Deployment
Canary Evaluation
Perf Regression Gate
RECOVERY ORCHESTRATION
Decision Engine
Policy Rules Engine
Risk Assessment
Action Prioritization
Alert Routing
Escalation Logic
SLO Breach Detection
Multi-signal Fusion
Runbook Selector
Human-in-loop Gate
Remediation
Auto-Restart / Rollback
Scale Out / In (HPA)
Traffic Rerouting
Pod Rescheduling (K8s)
Config Hot Reload
Circuit Breaker Trip
Blue-Green Flip
Chaos Validation
Audit Trail Emit
LLM COPILOT & RCA ENGINE
Components
LangChain / LlamaIndex Framework
Orchestrates LLM tool calls & agents
RAG Telemetry Pipeline
Indexes traces/logs → vector store (FAISS)
Automated Incident Post-Mortems
LLM-generated RCA reports on failure events
Natural Language Copilot Chat Widget
NL queries over live telemetry → Dashboard
RAG Data
INFRASTRUCTURE & PLATFORM LAYER
Kubernetes
Pods · Deployments · StatefulSets
HPA · Namespaces · RBAC
Node Affinity · PodDisruptionBudgets
CI/CD Pipeline
GitOps (ArgoCD / FluxCD)
Canary · Blue-Green · Shadow
GitHub Actions · Docker Builds
Config & Secrets
Consul / etcd · HashiCorp Vault
Feature Flags (LaunchDarkly)
OPA Policies · mTLS Certs
Security Layer
Zero-Trust Network Policy
SIEM Integration · Audit Logs
CVE Scanning · Falco Runtime
Data Stores
TimescaleDB · Redis · PostgreSQL
Vector Store (FAISS/Weaviate) · S3
Feature Store · Model Artefact Store
LEGEND
Healthy
Degraded
Failed
Telemetry Flow
Recovery Action
LLM / RAG Data Feed
Service Mesh
New in v2
HST · river ML · ADWIN streaming detection · YARP gateway · LLM Copilot & RCA Engine · RAG Telemetry Pipeline
CLOSED LOOP: Services → Telemetry → Observability → AI Prediction (HST/river) → Recovery Engine → Self-Heal → Services | LLM Copilot ← RAG ← Data Stores → Frontend


[Microservices Layer]
.NET APIs | Payment Service | Order Service | User Service
[Monitoring & Data Collection Layer]
Prometheus | Log Aggregation | Grafana
[AI Prediction Engine]
Anomaly Detection | Failure Prediction | Pattern Recognition
[Decision & Strategy Engine]
Recovery Strategy Selection | Severity Classification | Action Routing
[Execution Layer]
Kubernetes API | YARP API Gateway | Service Mesh
[LLM-Driven Observability Copilot & Post-Mortem Engine]
Conversational RAG Ingestion | Automated RCA Generation
[Presentation & Dashboard]
React UI Web Widgets / Chat Interface

8. Brief overview of the availability of necessary specialized domain expertise, knowledge, and data. (500 words max)

The success of this project hinges on expertise in three key areas: Distributed Systems (Kubernetes), Machine Learning, and Software Engineering (.NET). The team will acquire this expertise through a combination of existing skills, self-learning, and faculty guidance.

1. Gaining Specialized Domain Expertise:

- ***Microservices & Kubernetes:*** *Team members have completed academic modules covering cloud computing and distributed systems. Furthermore, they will supplement this by leveraging official Kubernetes documentation, industry-recognized certifications (e.g., CKAD course materials), and hands-on tutorials to deepen their practical understanding of cluster management, container orchestration, and API interactions.*
- ***AI for Systems:*** *The team has a background in machine learning principles. To specialize in time-series analysis and anomaly detection, they will undertake targeted online courses on platforms like Coursera (e.g., Deep Learning Specialization) and study papers on applying LSTMs and Isolation Forests to system metrics.*

2. Gaining Technical Knowledge & Skills:

- ***Technology Stack:*** *The project utilizes a modern technology stack (.NET 10, Python, React) which the team has experience with from prior academic projects. For new components (e.g., ML.NET, YARP), team members will dedicate the initial phase to rapid prototyping and following official tutorials and documentation.*
- ***Integration:*** *The most complex technical challenge is the integration between the ML engine (Python) and the .NET execution layer (C#). The team will build a robust API communication layer between these two components. Faculty guidance and online resources on microservice-to-microservice communication will be crucial for this.*

3. Data Collection and Availability:

- ***Data Source:*** *The primary challenge is obtaining labeled data of service failures to train the supervised ML models. To overcome this, the project will adopt a "data generation" approach:*
- ***Local Testbed:*** *A microservices application will be deployed on a local Kubernetes cluster.*
- ***Failure Injection:*** *The team will use tools like* ***Chaos Mesh*** *or a custom script to deliberately induce failures (e.g., high CPU load, memory leaks, network latency) in the test environment.*
- ***Metric & Labeling:*** *While failures are being injected, Prometheus will collect telemetry data. This data will be labeled with the time and type of failure injected, creating a labeled dataset for supervised learning.*
- ***Public Datasets:*** *As a supplement, the team will explore public datasets of system logs, such as those from the* ***LogHub*** *project or NASA's public prognostics datasets, to validate models on real-world data patterns.*

4. Ethical Clearance:

No, this project does not require ethical clearance. *The research will be conducted entirely in a controlled, synthetic, and isolated test environment. No human subjects will be involved, and no personal or sensitive user data will be collected or processed. All experiments will be performed on a local machine or a private cloud sandbox.*

9. Objectives and Novelty

Main Objective *To design, implement, and evaluate an open-source, self-healing framework that combines AI-driven failure prediction with automated, context-aware recovery strategies to significantly reduce downtime in microservices architectures.*			
Member Name with Registration No	*Sub Objective*	*Tasks*	*Novelty*
L.A.D.D.LiyanaArachchi IT23268494	*Context-Aware & Dependency-Driven Rule-Based Decision Engine*	*1. Build and maintain a live service dependency graph mapping inter-service dependencies.* *2. Implement context-aware rule evaluation that considers a service's full dependency context before firing a rule.* *3. Design a rule taxonomy for different decision contexts (isolated, upstream, cascading).* *4. Implement root cause localization logic using graph topology and anomaly timing.* *5. Design severity classification that incorporates dependency criticality (blast radius).* *6. Implement safety guardrails*	*The primary novelty is the **dependency-aware rule execution** that moves beyond static, service-isolated rules. This engine does not treat a service anomaly in isolation; it actively investigates the health of a service's upstream and downstream dependencies before executing a recovery action. This prevents actions like restarting a healthy service that is merely a symptom of a failing dependency, leading to smarter, more effective recoveries.*

		(cooldowns, max action frequency, escalation). *7. Log every decision with full context for analysis and dashboard display.* *8. Evaluate the engine's effectiveness against isolated rules in cascading failure scenarios.*	
Shalon Anton Fernando *IT22362544*	*Dynamic Model Updating with Concept Drift Detection*	*1. Design a model drift detection mechanism to monitor baseline assumption validity.* *2. Implement an online learning pipeline for incremental model updates.* *3. Design a model versioning and rollback system for safe updates.* *4. Implement concept drift detection algorithms (e.g., ADWIN, Page-Hinkley).* *5. Define and enforce model update triggers (time-based, drift-based, event-based).* *6. Evaluate the impact of dynamic updating on model accuracy against static models.*	*The genuine contribution is the **automated, continuous lifecycle management of anomaly detection models**. This component proactively addresses the "model decay" problem common in production AI systems. Instead of static models that degrade over time, this component enables the system to automatically adapt to new traffic patterns, business logic changes, and feature deployments, making the framework robust and viable for long-term, real-world use.*
A.G.L.M Wijesinghe *IT23264670*	*Adaptive Learning Decision Engine using Reinforcement Learning*	*1. Design an adaptive decision policy using reinforcement learning (e.g., Q-learning).* *2. Define the RL environment: state space, action space, and reward*	*The novel contribution is the **data-driven evolution of decision-making**. While the rule-based engine is fast and safe, it is static. This component introduces*

		function. *3. Implement a training pipeline using historical anomaly-action-outcome data.* *4. Design a safe exploration strategy (e.g., simulation or shadow mode).* *5. Implement a hybrid arbitration mechanism with the rule-based engine.* *6. Evaluate adaptive policy performance against the rule-based baseline.*	*intelligence that continuously learns from past actions. By using reinforcement learning, it can discover subtle, data-backed strategies that a human expert might not explicitly program, leading to improved long-term performance and a more sophisticated self-healing system.*
RUKSHAN E A Y *IT23263512*	*LLM-Driven Observability Copilot & Automated Post-Mortem Engine*	*1. Design and implement a RAG (Retrieval-Augmented Generation) pipeline for translating natural language to queries.* *2. Construct a context-aware chat widget for the dashboard for cross-functional stakeholders.* *3. Build an asynchronous listener to generate comprehensive Markdown post-mortem reports after recovery events.* *4. Define and implement systemic masking guardrails to scrub sensitive data.* *5. Evaluate LLM response quality using RAG evaluation frameworks and qualitative benchmarks.*	*The key innovation is enabling non-technical stakeholders (product owners, managers) to directly query and understand complex system health using natural language, without needing to understand PromQL, logs, or dashboards. Additionally, the system automatically generates professional, structured incident post-mortem reports immediately after every recovery event—a task that is typically manual, time-consuming, and often skipped. This combines generative AI with real-time observability to make operational intelligence accessible*

			and actionable for everyone, not just engineers.

10. This section is to be completed by the Supervisor and the Co-supervisor of the Project.

a) This research topic possesses a comprehensive scope suitable for a final-year project.

Yes	✓	No	

b) The proposed topic exhibits novelty.

Yes	✓	No	

c) The student group can successfully execute the proposed project.

Yes	✓	No	

d) The proposed sub-objectives reflect the students' areas of specialization.

Yes	✓	No	

e) Data required for the project is available.

Yes	✓	No	

f) Any other comments:

This is accepted.

	Title	First Name	Last Name	Signature
Supervisor*	Ms.	Hansi	De Silva	DHad.
Co-Supervisor*	Mr.	Eishan	Weerasinghe	
External Supervisor	Mrs	Archchana	Sindhujan	K. Archchana
Summary of external supervisor's (if any) experience and expertise				

*- At this stage, identifying the Co-Supervisor is compulsory. Students should reach out to a staff member, with guidance from their supervisor, to serve as the co-supervisor for this project.